sql/foreign_key_constraint_trips.sql

```
ALTER TABLE ONLY trips
ADD CONSTRAINT fk_trips_trip_requests
FOREIGN KEY (trip_request_id)
REFERENCES trip_requests(id);
```

Duplicate Constraints and Indexes



Duplicate constraints and Indexes can be added that cover the same columns as long as they have unique names. Be careful not to add duplicate Constraints or Indexes because these are consuming space, slowing down write operations. A Duplicate constraint or Index is redundant and one of them should be removed.

You've now covered how to use Foreign Key Constraints to link data from multiple tables together. Next you'll work with a less common constraint type called a CHECK constraint. This versatile constraint type allows you to capture conditional statements in their definition.

The Versatile Check Constraint

With a CHECK constraint you can go beyond checking for NULLs and connecting Primary Key and Foreign Key columns, and write a custom expression.

This constraint type gives you versatility in what expressions you write to help you enforce correctness in your data.

CHECK constraints are owned by a table. For Rails developers, if you've never seen a Check constraint, think of them as similar to Active Record Model Validations. Validations have some overlap in their purpose. Validations are created in Active Record models while Check constraints are created within the database for a table.

Model Validations express conditions that must be met for data to be considered valid.

Like all constraint types, Check constraints can be used on their own or in tandem with Active Record Validations.

Besides their primary purpose of enforcing correctness, Check constraints have another very useful purpose. Check constraints can be added as a placeholder due to their flexibility.

Some constraints require a table lock to be added without the option to check only new row modifications. Check constraints can be used to work around that. What does that look like?